



INTRODUCCIÓN A LAS BASES DE DATOS

Trabajo integrador

Grupo nro: 11

Integrantes: Martina Jazmín Medina, Sofía Castro y Franco Tomás Sacca

Docentes: Javier Vescio, Marcelo Wolf

Documentación de decisiones

Proyecto

Este documento registra las principales decisiones que tomamos para el diseño de la base de datos de un sistema para un restaurante.

El objetivo del modelo es representar la operatoria básica del restaurante: cuenta con mesas disponibles, mozos, turnos de trabajo, productos, pedidos realizados y detalle de los productos incluidos en cada pedido con sus cantidades.

Alcance funcional

Decidimos modelar un sistema orientado a la gestión de pedidos dentro del restaurante. Por eso, el alcance incluye:

- Registro de mesas, capacidad, sector y observaciones.
- Registro de mozos y el turno en el que trabajan.
- Registro de categorías de productos.
- Registro de productos, precios y disponibilidad.
- Registro de pedidos asociados a una mesa y a un mozo.
- Registro del detalle de cada pedido, con cantidades, precios unitarios y observaciones.
- Consultas para obtener información útil para el negocio y reportes simples.

Motor y estilo SQL

Usamos distintas restricciones:

- Usamos AUTO INCREMENT para generar identificadores primarios.
- Usamos ENUM para restringir los estados posibles de un pedido.
- Usamos TINYINT(1) para representar la disponibilidad de productos.
- Usamos DATETIME DEFAULT CURRENT_TIMESTAMP para registrar automáticamente la fecha y hora de creación de un pedido.

La elección permite trabajar con una sintaxis simple, conocida y suficiente para el nivel del proyecto.

Identificadores

Todas las tablas principales usan claves primarias:

- id_mesa
- id_turno
- id_mozo
- id_categoria
- id_producto
- id_pedido
- id_detalle

Durante la carga de datos con **INSERT**, se decidió incluir manualmente los valores de las claves primarias, por ejemplo, **id_pedido** e **id_producto**, aunque las tablas tengan activada la propiedad **AUTO_INCREMENT**.

Esta decisión nos permite asegurar que las líneas de '*detalles_pedido*' apuntan con precisión a los pedidos y productos correspondientes durante la carga inicial de datos de prueba. De esta forma, se evitan inconsistencias si el orden de inserción cambia o si existe algún fallo durante la ejecución del script.

Separación de entidades

El modelo separa la información en entidades independientes para evitar duplicación y mantener la base ordenada:

- **mesas**: almacena la información física o lógica de cada mesa.
- **turnos**: almacena las jornadas disponibles.
- **mozos**: almacena los empleados que atienden pedidos.
- **categorías**: organiza los productos del menú.
- **productos**: almacena los platos y bebidas disponibles.
- **pedidos**: representa cada pedido realizado.
- **detalles_pedido**: representa los productos incluidos en cada pedido.

Relaciones entre tablas

Se definieron claves foráneas para representar las relaciones principales, incluidas dentro del modelo conceptual:

Estas relaciones aseguran integridad referencial: nos aseguramos de que no exista un pedido asociado a una mesa inexistente, ni un detalle de pedido asociado a un producto inexistente.

Además, el orden de los bloques *CREATE TABLE* se organizó de manera secuencial. Primero se crean las entidades independientes, como *'mesas'*, *'turnos'* y *'categorias'*, y después las entidades dependientes, como *'mozos'*, *'productos'*, *'pedidos'* y *'detalles_pedido'*. Esta decisión que tomamos evita conflictos al crear restricciones con FOREIGN KEY, porque cada tabla referenciada existe antes de ser utilizada por otra.

Tabla de detalle de pedido

Resolución de la Relación Muchos a Muchos (N:M) entre Pedidos y Productos: un pedido contiene 1 o más productos, y un producto puede estar en varios pedidos, es por eso que se creó la tabla intermedia o entidad asociativa *'detalles_pedidos'*, esto cumple con la Tercera Forma Normal (3FN).

Esta tabla resuelve la relación muchos a muchos entre *pedidos* y *productos*, y además guarda información propia de cada línea del pedido:

- cantidad
- precio_unitario
- observaciones

Guardar el precio unitario en el detalle permite conservar el precio cobrado en el momento del pedido, aunque el precio actual del producto cambie después en la tabla *productos*.

Estados del pedido

El campo **estado** de la tabla *'pedidos'* se definió como *ENUM* con los valores:

- Pendiente
- En preparación
- Entregado
- Cancelado
- Cerrado

Esta decisión limita los valores posibles y evita estados escritos de formas diferentes. Los estados permiten diferenciar pedidos activos, finalizados o anulados, lo cual es necesario para consultas de gestión y reportes de ventas.

Disponibilidad de productos

La tabla *productos* incluye el campo **disponible**, representado con **TINYINT (1)**.

Decidimos mantener los productos no disponibles dentro de la tabla en lugar de eliminarlos. Esto nos permite conservarlos en el historial, seguir relacionándolos con pedidos anteriores y reactivarlos si vuelven a estar disponibles.

Implementamos **TINYINT(1) DEFAULT 1** ya que, este atributo representa un valor booleano: disponible o no disponible. El valor por defecto en **1** asegura que cualquier nuevo ítem cargado al menú quede activo automáticamente.

Registro automático de fecha y hora

El atributo *fecha_hora* de la tabla '*pedidos*' se definió con DATETIME DEFAULT CURRENT_TIMESTAMP.

Para que cada pedido pueda guardar automáticamente el momento exacto del servicio sin poner una carga manual de fecha y hora.

Tipo de datos

Los precios se definieron con *DECIMAL(10,2)* tanto en *productos.precio_actual* como en *detalles_pedido.precio_unitario*.

Se rechazó el uso de tipos enteros como ``INT``, ya que los precios pueden necesitar centavos o valores con dos decimales.

Uso de Tipos de Datos "TEXT" para mayor flexibilidad: Se necesita registrar información descriptiva que no sigue un formato estándar, como las notas sobre las mesas o los platos. Se utilizó el tipo de dato TEXT para campos como '*observaciones*' en las tablas '*mesas*' y '*detalles_pedidos*', y para '*descripción*' en '*productos*' y '*categorías*', en lugar de un VARCHAR con límite corto

Categorías del menú

Los productos se agrupan en categorías como entrada, principal, postre, bebida y alcohol.

Esta decisión facilita:

- Filtrar productos por tipo.
- Calcular precios promedio por categoría.
- Generar reportes de ingresos por categoría.
- Mantener un menú más ordenado.

La categoría se modeló como tabla independiente para evitar repetir el nombre de la categoría en cada producto y mantener la normalización.

Mesas y sectores

Las mesas tienen número, capacidad, sector y observaciones.

Se incluyó el sector para diferenciar ubicaciones del restaurante, como salón interior, terraza o salón VIP. Esto permite analizar pedidos o facturación por zona.

El campo *observaciones* se usó para registrar información flexible sobre la mesa, por ejemplo, si está libre u ocupada. Para una versión más avanzada, ese dato podría normalizarse en un campo de estado específico.

Turnos y mozos

Los turnos se modelaron en una tabla independiente y los mozos se relacionan con ella.

Esta decisión evita repetir textos como "Mañana", "Tarde" o "Noche" en cada registro de mozo. También permite consultar fácilmente que mozos trabajan en cada turno.

Datos de prueba

En el script **TP_BD1.sql** agregamos inserciones de prueba para testear la base.

- Los datos fueron pensados para cubrir distintos escenarios: pedidos entregados, cancelados, cerrados y en preparación; productos disponibles y no disponibles; diferentes mesas, sectores y mozos.
- Contamos con un set de datos suficientemente complejo para probar filtros con *WHERE*, ordenamientos con *ORDER BY*, agrupamientos con *GROUP BY*, funciones de agregación y consultas con distintos tipos de *JOIN*.
- También se incluyeron ejemplos de modificación y eliminación de datos mediante *UPDATE* y *DELETE*, para demostrar operaciones básicas sobre la base.

Consultas

El archivo `consultas.sql` incluye consultas orientadas a recuperar información útil del sistema.

Se cubren consultas de:

- Listado de mozos por turno.
- Productos por categoría.
- Pedidos con mesa y mozo.
- Detalles de pedido con categoría y producto.
- Total a pagar por pedido.
- Productos más pedidos.
- Cantidad de pedidos por estado.

- Total facturado por mesa.
- Categoría con mayor ingreso.
- Ranking de mozos por ventas.
- Productos que nunca fueron pedidos.

Estas consultas demuestran el uso de *JOIN*, *LEFT JOIN*, subconsultas, *funciones de agregación*, *GROUP BY*, *ORDER BY* y *filtros por fecha o estado*.

Normalización

El modelo sigue una estructura normalizada:

- No se repiten datos de categorías dentro de productos.
- No se repiten datos de turnos dentro de mozos.
- No se repiten datos de productos dentro de cada pedido.
- El detalle del pedido se separa del encabezado del pedido.